RAPPORT D'AUDIT — USAGE INTERNE

ChainSentinel

Audit stratégique, technique et fonctionnel

Plateforme open source d'intelligence blockchain, de surveillance de portefeuilles et de détection de comportements suspects

Date	6 novembre 2026
Périmètre audité	Commit `f34b31d` (branche `main`), copie figée par `git archive`
Hors périmètre	Travaux de la « phase 2 » en cours, non committés au moment de l'audit (Rule Builder, `/dashboard`, scénarios de démo, test de bout en bout, en-têtes webhook)
Méthode	Lecture intégrale du code et des tests ; exécution des builds et tests ; scénarios exécutés contre les classes compilées ; appels réels à l'API démarrée ; recherche sur sources officielles pour le marché
Licence du projet	Apache-2.0

> **Recommandation finale : continuer, en repositionnant et en simplifiant.** ChainSentinel ne doit pas viser un « Chainalysis open source » : la valeur de ce marché est dans des données d'attribution propriétaires que le projet n'a pas. Sa place crédible est celle d'une **plateforme self-hosted de surveillance on-chain programmable et de scoring explicable**, centrée d'abord sur TRON et Ethereum et sur les stablecoins USDT et USDC. Le moteur de règles est le meilleur actif du dépôt. En revanche, la fiabilité des données, la persistance et la livraison réelle des notifications doivent être réglées avant toute nouvelle fonctionnalité.

Conventions : **FAIT** = vérifié dans le code ou par exécution ; **HYPOTHÈSE** = plausible, non démontré ; **RECO** = recommandation. Les chemins de fichiers sont relatifs à la racine du dépôt. Aucun chiffre de marché n'est inventé : « Non communiqué » ou « À vérifier » signale l'absence de source publique.

Sommaire

1. Résumé exécutif
2. Évaluation globale et notes
3. État réel du code
4. Analyse du marché
5. Benchmark concurrentiel
6. Blockchains supportées
7. Audit des détecteurs
8. Audit du risk scoring
9. Audit du wallet intelligence
10. Audit du Programmable Alerting Engine
11. Audit de l'architecture
12. Audit de sécurité
13. Données et threat intelligence
14. Analyse UX/UI
15. Open source et modèle économique
16. Architecture cible recommandée
17. Matrice des priorités P0 à P3
18. Feuille de route
19. Risques majeurs
20. Recommandation finale et dix décisions

1. Résumé exécutif

ChainSentinel est un monolithe modulaire Java 21 / Spring Boot 4 (modules `oci-core`, `oci-blockchain`, `oci-intelligence`, `oci-alerting`, `oci-app`) avec une console Angular 22. Le dépôt est propre, bien découpé et abondamment documenté (12 ADR, contrat d'API, guides). Les builds passent : **323 tests backend** (core 3, blockchain 46, intelligence 51, alerting 196, app 27) et **53 tests frontend**, tous verts (FAIT, exécuté le 6 novembre 2026).

Mais la documentation décrit un produit plus avancé que le code. Les constats déterminants :

- **Données incomplètes.** Seuls les événements `Transfer` ERC-20/TRC-20 sont indexés (`BlockIndexer.poll` n'appelle que `getTokenTransfers`, ligne 91). Les transferts natifs (ETH, TRX), les approvals, les événements d'administration de contrats et les gels USDT/USDC ne sont jamais lus. Deux détecteurs ne peuvent donc jamais se déclencher sur des données réelles.
- **Score manipulable à coût quasi nul.** Un transfert de 0,000001 USDT, ou un faux token, depuis une adresse sanctionnée donne au destinataire un score 80/HIGH (`ExposureDetector.direct` : aucun seuil de montant, aucun filtre sur les tokens connus). Démontré par exécution.
- **Aucune persistance.** Règles, alertes, clés, webhooks, profils et scores vivent en mémoire et sont perdus au redémarrage. PostgreSQL démarre dans Docker Compose mais n'est jamais utilisé.
- **Notifications inopérantes hors démo.** Les webhooks créés par l'API reçoivent un identifiant `wh_…` que le validateur de règles refuse (`RuleValidator` ligne 442, motif slug). Les connecteurs Slack, Discord, Telegram et email existent comme classes mais ne sont jamais instanciés.
- **Trois dialectes de règles incompatibles** : backend `openchain.io/rule/v1`, documentation `oci.rules/v1`, frontend `openchain.io/v1`. Seul le premier est accepté.
- **Écart documentation / code.** Le README présente comme disponibles l'isolation par tenant, les clés d'idempotence, le rate limiting, le journal d'audit, les métriques Prometheus et les traces OpenTelemetry : aucun n'existe dans le code.

Points solides : montants exacts (`BigInteger` / `BigDecimal`), décimales USDT/USDC correctes, gestion des réorganisations de blocs testée, transport RPC résilient, moteur de règles (fenêtres en temps d'événement, séquences, déduplication, cooldown, retry, dry-run), protections SSRF, YAML et ReDoS de niveau production, clés API correctement hachées, chaque point du score tracé.

Sur le marché, les leaders (Chainalysis, TRM, Elliptic, Crystal, Merkle Science) vendent des données d'attribution propriétaires couvrant 65 à 330+ chaînes, sans prix publics. Le self-hosting est quasi absent chez eux (seul exemple public : Breadcrumbs « Data on Premises », 380 000 $/an + 30 000 $ d'installation). Le concurrent open source le plus direct est **OpenZeppelin Monitor (AGPL-3.0)**, qui a remplacé Defender, retiré le 1er juillet 2026. C'est le créneau à viser.

2. Évaluation globale et notes

Domaine	Note	Justification (preuves dans le dépôt)
Maturité technique	**4/10**	Bibliothèques soignées et testées, mais tout en mémoire (`ModulesConfiguration`), aucune migration, aucune observabilité, notifications non livrables hors démo, une large part du catalogue de champs jamais alimentée.
Qualité de l'architecture	**6/10**	Monolithe modulaire propre : domaine sans framework, dépendances unidirectionnelles vérifiées dans les POM, SPI `BlockchainProvider`. Pénalisé par un pipeline entièrement sérialisé (`synchronized`), un pool de 2 threads partagé entre ingestion et notifications, trois DSL concurrents.
Couverture blockchain	**3/10**	4 réseaux sur 10 ont un adaptateur (Ethereum, BSC, Polygon, TRON), tous limités aux transferts de tokens. Arbitrum, Optimism, Base et Avalanche absents de l'enum `Network` ; Bitcoin et Solana y figurent sans adaptateur. `FailoverProvider` testé mais non branché.
Qualité des données	**3/10**	Montants exacts, mais natifs absents, tokens inconnus traités en décimales 0 et mêlés aux flux, transferts falsifiables acceptés, adresses non validées (`Address`), rétractations de reorg ignorées par `IntelligencePipeline`, labels uniquement fictifs, fixtures de test écrites à la main.
Détection	**3/10**	10 détecteurs déterministes et explicables (25 tests dans `DetectorsTest`) ; 5 scénarios typologiques sur 17 couverts, 7 absents ; seuils non calibrés ; faux positifs démontrés (aller-retour via un exchange, wallet déjà approvisionné).
Explicabilité du scoring	**5/10**	Chaque point est attribué à une `Contribution` avec source, poids, confiance et âge (`RiskScoringEngineTest`, 11 tests). Mais les garanties de l'ADR-0008 ne sont pas tenues : deux labels de source non autoritative donnent 89/CRITICAL, aucun recalcul quand un label change, double comptage entre facteurs.
Wallet intelligence	**3/10**	Profils exacts et exposition sur 3 sauts temporellement cohérente, mais ni clustering, ni taint quantitatif, ni graphe persistant (ADR-0005 en documentation seulement) ; chaque événement rebalaye tout l'historique de l'adresse.
Alerting	**5/10**	Moteur 7/10 : fenêtres exactes, séquences, gestion des reorgs, déduplication, cooldown, backoff, dead letter, dry-run, tests embarqués. Chaîne de bout en bout 3/10 : aucune livraison réelle possible, séquences limitées à une clé, pas de franchissement de seuil.
Sécurité	**5/10**	Clés API HMAC-SHA256 à comparaison en temps constant, permissions par endpoint, erreurs génériques, CSP, SSRF/YAML/ReDoS exemplaires. Manquent : rate limiting, journal d'audit persistant et complet, bornes mémoire ; rôles utilisateurs sans effet ; jeton stocké dans le `localStorage` ; défauts démo exposés dans Compose.
Expérience utilisateur	**3/10**	Console riche en apparence, accessibilité soignée, graphe Cytoscape. Mais Rule Builder produisant un format refusé, mode mock commutable même en build de production, preuves d'alerte non affichées, ni export de rapport ni dossier d'investigation.
Potentiel commercial	**4/10**	Créneau réel (self-hosted, souveraineté, monitoring programmable, TRON/USDT) mais pas de données d'attribution, et des acheteurs régulés qui privilégient des fournisseurs reconnus. Acheteurs plausibles : protocoles DeFi, équipes risque, petits PSP, acteurs exigeant l'on-premise.
Compétitivité open source	**5/10**	Apache-2.0, fichiers communautaires complets (DCO, sécurité, modèles d'issues), moteur de règles différenciant. Face à OpenZeppelin Monitor (mature, AGPL, actif) et GraphSense (MIT, investigation), il faut un périmètre plus net et des données ouvertes fiables.

3. État réel du code

Statuts : **Implémenté+testé** **Partiel** **Docs seulement** **Simulé/fictif** **Absent**.

Fonctionnalité	Statut	Preuve
Adaptateur EVM (Ethereum)	Partiel	`EvmJsonRpcProvider` : `Transfer` ERC-20 via `eth_getLogs`, reçus, soldes ; 8 tests sur JSON écrit à la main.
BSC, Polygon	Partiel, non testé	Même adaptateur, aucun test dédié.
TRON	Partiel	`TronGridProvider` : `Transfer` TRC-20 ; adresses Base58Check testées avec des vecteurs réels.
Arbitrum, Optimism, Base, Avalanche	Absent	Absents de `oci-core/.../model/Network.java`.
Bitcoin, Solana	Absent	Constantes d'enum sans adaptateur.
Indexation, confirmations, reorgs	Implémenté+testé	`BlockIndexer`, 9 tests (reorg 1, 3, égale à la fenêtre, au-delà → `ReorgTooDeepException`) ; uniquement sur `SimulatedChainProvider`.
Checkpoint durable	Absent	Seul `InMemoryCheckpointStore` existe.
Transferts natifs, transactions internes, approvals, gels de stablecoins	Absent	Seul `TRANSFER_TOPIC` est décodé (`EvmCodec`) ; aucune occurrence de « blacklist » ou « freeze » dans le code de production.
Transport résilient (retry, circuit breaker, rate limit)	Implémenté+testé	`HttpJsonTransportTest` (serveur HTTP local), `ResiliencePrimitivesTest`. `Retry-After` plafonné à 5 s.
Détecteurs (10)	Implémenté+testé, 2 inatteignables	`oci-intelligence/.../detection/detectors` ; `UnusualApprovalDetector` et `OwnershipChangeDetector` sans producteur d'événements.
Scoring explicable	Partiel	`RiskScoringEngine` ; écarts avec l'ADR-0008 (§8).
Labels / threat intelligence	Simulé/fictif	`labels/demo-labels.csv` : 5 entrées fictives aux adresses invalides ; aucune source réelle.
Graphe persistant (ADR-0005)	Docs seulement	Aucune table, aucun code.
DSL de règles, AND/OR/NOT, fenêtres	Implémenté+testé	`RuleValidator`, `RuleCompiler`, `SlidingWindow` ; 23 + 17 tests.
Séquences / corrélation	Partiel	`SequenceState` : une seule clé pour toutes les étapes.
Dry-run, déduplication, cooldown, retry, dead letter	Implémenté+testé	`RuleTester`, `EventDedupCache`, `AlertService`, `DeliveryService`.
Escalade	Partiel	`EscalationPolicy` : hausse de priorité seulement, aucune re-notification.
Webhooks signés + garde SSRF	Partiel	`WebhookSignature`, `SsrfGuard`, `SafeHttpClient` testés ; en-têtes et payload différents de la doc ; non référençables par une règle.
Slack, Discord, Telegram, email, push	Partiel (classes seulement)	Jamais enregistrés dans `ConnectorRegistry` par l'application.
API REST v1	Implémenté+testé	Contrôleurs `oci-app/.../api`, 27 tests ; appels réels vérifiés (§12).
Clés API, permissions	Implémenté+testé	`ApiKeyService`, `SecurityConfiguration` ; 401, 403, révocation, clé dans l'URL refusée vérifiés par appels réels.
OIDC	Partiel, non vérifié	Configuration conditionnelle ; non testé avec un fournisseur d'identité réel.
Persistance, idempotence d'API, rate limiting, ETag/If-Match	Absent	`TODO(persistence)`, `TODO(concurrency)` dans `oci-app`.
Multi-tenancy	Docs seulement	Aucune occurrence de tenant dans le code de production.

Bus d'événements (Kafka), outbox	Docs seulement	ADR-0006 ; appels directs en mémoire.
Observabilité (Prometheus, OpenTelemetry)	Docs/infra seulement	Aucun registre de métriques dans les POM ; le tableau de bord Grafana interroge 10 métriques inexistantes.
Console Angular	Partiel	12 routes appelant l'API réelle ; Rule Builder cassé ; mock activable en production (`settings.ts`).
CI GitHub Actions	Implémenté	`.github/workflows/ci.yml` : build Maven, frontend, images, CodeQL ; contrôle de licences non bloquant.

4. Analyse du marché

4.1 Moteurs de la demande

La demande payante est tirée par la réglementation : Travel Rule européenne (règlement 2023/1113, applicable depuis le 30 décembre 2024), MiCA, AMLR (application annoncée au 10 juillet 2027, à vérifier au Journal officiel), obligations BSA/FinCEN et OFAC aux États-Unis, et GENIUS Act pour les émetteurs de stablecoins (proposition de règle FinCEN/OFAC publiée le 10 avril 2026, non définitive). Ces textes poussent les entités régulées vers des fournisseurs reconnus, défendables devant un régulateur. Un outil open source y est un **complément** (règles internes, surveillance de trésorerie), rarement un remplaçant.

4.2 Problèmes métier et apport possible de ChainSentinel

Problème	Utilisateurs	Données nécessaires	Méthode	Apport de ChainSentinel	Limites / faux positifs
Wallets suspects	CEX, PSP, compliance	Labels attribués, historique	Exposition + score	Score explicable, self-hosted	Sans labels réels, aucune valeur
Flux inhabituels	Risk teams, PSP	Historique par adresse	Statistiques robustes (MAD)	Détecteur existant, règles	Paie, bonus, ventes ponctuelles
Traçage multi-wallets	Enquêteurs	Graphe complet, natifs, internes	BFS borné, taint	3 sauts amont aujourd'hui	Pas de taint quantitatif ; les exchanges coupent la piste
Fonds vers adresses à risque	CEX, PSP	Sanctions, scams	Exposition directe/indirecte	Règles + webhooks	Dusting à filtrer (aujourd'hui exploitable)
Layering	Compliance, LE	Chaînes de relais, swaps	Chemins multi-sauts rapides	Partiel (fast-forwarding)	Routeurs DEX, agrégateurs
Dispersion / consolidation	Compliance	Contreparties distinctes	Fan-out / fan-in fenêtrés	Fan-out existant ; fan-in absent	Hot wallets, payroll, airdrops
Contrats suspects	DeFi, wallets	Appels, approvals, bytecode	Décodage + labels contrats	Absent (événements non indexés)	Approvals illimités légitimes
Fonds de hacks / scams	CEX, LE	Adresses d'incidents	Exposition + watchlists	Règles watchlist possibles	Listes publiques lentes et partielles
Phishing	Wallets, DeFi	Domaines, adresses look-alike	Listes + similarité d'adresses	Absent	Hors cœur actuel (latence pré-transaction)
Mixers et bridges	Compliance	Labels de contrats, natifs	Exposition	Partiel (dépôts ETH invisibles)	Bridges légitimes majoritaires

Wallets de grande valeur	Risk, trésoreries	Soldes réels, seuils fiat	Règles de seuil	Règles exprimables (USDT)	Pas de prix USD
Changement de comportement	Risk teams	Historique long	Baseline par adresse	Détecteurs simples	Historique perdu au redémarrage
Temps réel	Tous	Indexation fiable	Streaming + règles	Moteur prêt	Ingestion non persistée, sérialisée

4.3 Qui adopte, qui paie

Adoption probable en open source : protocoles DeFi (anciens utilisateurs de Defender), équipes risque et trésoreries, petits PSP sur TRON/USDT, chercheurs, intégrateurs. Paiement plausible : acteurs exigeant l'on-premise ou la souveraineté (créneau illustré par l'offre Breadcrumbs à 380 000 $/an), PSP voulant un pack « sanctions + gels + règles » prêt à l'emploi, intégrateurs achetant du support. Les CEX et banques paieront surtout pour des données d'attribution, que le projet n'a pas.

5. Benchmark concurrentiel

✓ documenté par l'éditeur ; ~ partiel ou marketing seulement ; ✗ absent ; NC non communiqué. Sources dans le rapport de recherche (pages officielles consultées le 6 novembre 2026).

Critère	ChainSentinel (FAIT)	Chainalysis	TRM	Elliptic	Breadcrumbs	Arkham	Blockaid	OZ Monitor (OSS)
Exploration	~ API wallet/tx, données simulées en démo	✓	✓	✓	✓	✓	✗	✗
Wallet intelligence (labels)	✗ aucun label réel	✓	✓	✓	✓	✓	~	✗
Traçage	~ 3 sauts amont, tokens seulement	✓	✓	✓ cross-chain	✓	✓	✗	✗
Graphe	~ Cytoscape, non persistant	✓	✓	✓	✓	✓	✗	✗
Détection AML	~ 5/17 typologies	✓ KYT	✓	✓	✓	~	~	✗
Scams	✗	✓	✓ Chainabuse	~	~	✗	✓	✗
Smart contracts malveillants	✗ (événements non indexés)	✓ Hexagate	~	~	✗	✗	✓	~ règles manuelles
Risk scoring	~ explicable, manipulable	✓	✓	✓	✓	✗	✓ verdict	✗
Threat intelligence	✗	✓	✓	✓	~	~	✓	✗
Alertes programmables	✓ moteur YAML ; ✗ livraison hors démo	~ / ✓ Gatelang	✓	✓	✓	~	~	✓

API publique	✓ REST v1 (sans persistance)	~ sous contrat	✓ docs	À vérifier	✓	✓ sur candidature	✓	n/a
Intégration B2B	~ webhooks seulement	✓	✓	✓	✓	~	✓	~
Open source	✓ Apache-2.0	✗	✗	✗	✗	✗	✗	✓ AGPL-3.0
Self-hosted	✓ Docker Compose	NC	NC	NC	✓ 380 k$/an	✗	NC	✓
Coût d'exploitation	Infrastructure + RPC	NC (devis)	NC ; API sanctions gratuite	NC	0 à 380 k$/an	API dès 100 $/mois	NC	Infrastructure + RPC

Autres acteurs pertinents : Crystal Intelligence (330+ chaînes annoncées), Scorechain (cloud privé UE, sans on-premise), AMLBot (fort sur TRON/USDT, modèle à crédits), Merkle Science (domaine correct `merklescience.com` ; `merkle.io` sert un produit MEV sans rapport), Hypernative et Webacy (sécurité on-chain), Nansen (analytics, prix publics), GraphSense (open source MIT, investigation). Alchemy n'est pas un concurrent : c'est une brique d'ingestion (RPC et webhooks d'activité d'adresses) que ChainSentinel pourrait utiliser.

Lecture. ChainSentinel ne gagne sur aucun critère « données ». Il peut gagner sur la combinaison *open source* + *self-hosted* + *règles programmables* + *explicabilité*, aujourd'hui occupée seulement par OpenZeppelin Monitor, qui n'a ni scoring de risque ni wallet intelligence.

6. Blockchains supportées

Réseau	État (FAIT)	Modèle et spécificités	Points d'attention
Ethereum	Partiel	Compte/EVM ; ERC-20 via logs ; finalité par checkpoint (tag `finalized` lu mais non utilisé par l'indexer).	Natifs et transactions internes (traces) indispensables : mixers ETH invisibles aujourd'hui.
TRON	Partiel	Compte, TRC-20 via `gettransactioninfobyblocknum` ; 19 confirmations ≈ solidification.	Réseau n°1 de l'USDT : gels Tether (`AddedBlackList`) à traiter en priorité ; TRX et TRC-10 absents.
BSC, Polygon	Partiel, non testé	EVM ; USDT/USDC BSC en 18 décimales (correct dans le registre).	Tests dédiés manquants ; Polygon : fenêtre de reorg 256.
Arbitrum, Optimism, Base	Absent	EVM L2 : finalité liée à L1, séquenceur centralisé.	Ajout peu coûteux (même adaptateur) : enum, normalisation d'adresse, registre de tokens, règles de finalité L2.
Avalanche C-Chain	Absent	EVM, finalité rapide.	Idem.
Bitcoin	Absent	UTXO : un autre modèle de flux (entrées/sorties multiples, heuristiques de change).	Nouvel adaptateur et nouveau modèle de flux ; ne pas l'ajouter avant que la couche EVM/TRON soit fiable.
Solana	Absent	Comptes et programmes, SPL tokens, très gros débit.	Coût d'indexation élevé ; dernier de la liste.

Stratégie progressive (RECO). Étape 1 : rendre TRON et Ethereum complets (natifs, approvals, gels USDT/USDC, persistance, fixtures enregistrées depuis de vrais nœuds). Étape 2 : BSC et Polygon testés, puis Arbitrum, Base,

Optimism et Avalanche via l'adaptateur EVM. Étape 3 : Bitcoin. Étape 4 : Solana. Pour les stablecoins, priorité aux événements de gel et de déblocage, à l'émission et au rachat, et aux seuils exprimés en valeur fiat.

Coûts d'infrastructure (HYPOTHÈSE). Les RPC publics sont limités en débit ; un nœud Ethereum self-hosted (reth, erigon) ou un fournisseur payant devient nécessaire au-delà de quelques adresses surveillées. TronGrid impose des quotas par clé. Le budget doit être chiffré par réseau avant tout engagement de couverture.

7. Audit des détecteurs

Détecteur	Logique et seuils par défaut	Faux positifs	Contournement	Statut
UnusualAmount	z robuste (médiane, MAD) ≥ 3,5 ; ≥ 10 flux d'historique	Paie, premier gros retrait ; déclenché aussi côté receveur	Montée progressive des montants	Testé
Structuring	≥ 3 sorties dans [9 000 ; 10 000) sur 24 h ; seuil unique pour tout token	Paiements récurrents ; seuil absurde hors stablecoins	Montants plus bas, plusieurs wallets	Testé
FastForwarding	Sorties ≥ 90 % des entrées en 30 min	Solde préexistant ignoré (démontré) ; adresses de dépôt d'exchange	Délai > 30 min, swap	Testé, FP élevés
DormantReactivation	Inactivité ≥ 180 j	Cold wallets	—	Quasi inopérant : exige 180 j d'historique en mémoire
RapidDrain	Sorties ≥ 80 % du solde estimé en 1 h	Migration vers cold storage	Drain lent, multi-tokens	Testé ; solde estimé ≠ solde réel
FanOut	≥ 20 destinataires distincts en 1 h	Hot wallets, payroll, airdrops (aucune exclusion EXCHANGE)	19 destinataires	Testé ; type mal nommé (`FLOW_CONCENTRATION`)
CircularFlow	Cycle ≤ 4 flux en 7 j, même token	Aller-retour via un exchange (démontré)	Swap, > 3 intermédiaires	Testé, FP élevés
Exposure (direct)	Toute contrepartie labellisée, sans seuil	Dusting, faux tokens (démontré : 80/HIGH)	—	**Vulnérable**
Exposure (indirect)	BFS amont ≤ 3 sauts, confiance × 0,5 par saut	1 USDT teinté dans 1 M → HIGH	Passage par exchange/bridge	Testé, non proportionnel
UnusualApproval, OwnershipChange	Approval ≥ 296−1 ; changement d'admin	Approvals illimités usuels	Permit/Permit2	Inatteignables en réel

Couverture des 17 scénarios

Statut	Scénarios
Couverts (logique testée, non calibrée)	Structuring, fan-out, rapid movement, circular transactions, sanction exposure (logique seulement : aucune liste réelle)
Partiels	Layering, dormant wallet, mixer interaction, scam exposure, drainer / exploitation de contrats
Absents	Fan-in, bridge hopping, dusting, address poisoning, wash trading, sybil, MEV et bots

Anomalie, suspicion, preuve. Le code distingue déjà `EvidenceLevel` (ANOMALY, RISK_SIGNAL, CONFIRMED_BY_SOURCE) et emploie un vocabulaire prudent (« exposure », « risk signal »). Mais le moteur de score

n'exploite pas ce niveau : une simple anomalie statistique et une désignation officielle pèsent dans le même score sans plafond. C'est à corriger avant toute communication publique.

Tests manquants (RECO). Jeu de cas réels étiquetés avec mesure de précision et de rappel par détecteur ; cas légitimes qui ne doivent pas déclencher (exchange, payroll, bridge, DEX) ; scénarios adversariaux (dust, faux token, address poisoning) ; absence de signal sur flux rétracté ; performance sur adresses « hub ».

8. Audit du risk scoring

Formule réelle (FAIT, `RiskScoringEngine.score`). Pour chaque élément : q = confiance × multiplicateur de sévérité × 0,5âge/180 j. Agrégation « noisy-OR » à l'intérieur de chaque facteur, plafonnée à son poids, puis entre facteurs ; score = 100 × S arrondi. Seul le label SANCTIONED ne décroît pas. Bandes : LOW < 35 ≤ MEDIUM < 65 ≤ HIGH < 85 ≤ CRITICAL.

Engagement de l'ADR-0008	Code	Statut
Maximum par facteur	Noisy-OR	Écart
Une anomalie seule ne dépasse pas MEDIUM ; seul CONFIRMED_BY_SOURCE atteint CRITICAL	`evidenceLevel` ignoré ; 2 labels d'un blog → 89/CRITICAL	**Non tenu**
L'exposition directe supplante l'indirecte	Non	Non tenu
Recalcul en cas de rétractation (reorg)	Rétractations ignorées	Non tenu
Historique en table `risk_scores`	Mémoire (500 entrées par adresse)	Docs seulement

Modèle recommandé. Séparer quatre scores au lieu d'un : *risk score* (décision, plafonné par le niveau de preuve), *confidence score* (qualité et autorité des sources, complétude des données), *exposure score* (montant et part des fonds exposés, par saut, avec taint pro-rata ou haircut), *anomaly score* (comportemental, jamais au-dessus de MEDIUM seul). Règles anti-double-comptage : déduplication par transaction entre facteurs, primauté direct sur indirect, un signal par fenêtre et par adresse. Contre les informations anciennes ou non vérifiées : décroissance obligatoire pour toute source non officielle, date de vérification et de validité par label, refus de dater par défaut à l'époque Unix (aujourd'hui un label sans date est silencieusement annulé, ou éternel s'il est SANCTIONED). Recalcul : file de rescoring déclenchée par l'ajout, le retrait ou le délistage d'un label, par un changement de configuration et par une rétractation, avec version du modèle dans chaque score.

9. Audit du wallet intelligence

Capacité attendue	État	Commentaire
Recherche d'adresse, historique	Implémenté	API et console ; historique limité à ce qui a été indexé en mémoire.
Contreparties, flux entrants/sortants	Implémenté	`WalletProfileAccumulator` : agrégats exacts par token, top 10 des contreparties.
Navigation dans le graphe	Partiel	Profondeur ≤ 3, non persistant, pas d'annotation ni de sauvegarde.
Clusters, comportements similaires	Absent	Aucune heuristique (financeur commun du gas, adresses de dépôt, co-dépense UTXO).
Propagation du risque	Partiel	Chemin textuel dans les preuves, sans objet graphe structuré.
Exposition directe et indirecte	Partiel	Pas de montant exposé ni de pourcentage.
Explication de la provenance du score	Implémenté	Contributions tracées, mais non affichées en triage d'alerte.

Limites de l'attribution. Des données publiques permettent d'établir des liens (financement commun, dépôts vers une même adresse d'exchange), jamais une identité. Un cluster est une hypothèse avec un niveau de confiance, pas une entité : l'interface doit l'afficher comme tel, avec les heuristiques ayant produit le regroupement.

Stratégie multi-hop (RECO). Profondeur par défaut 2, maximum 4 ; budget de nœuds et d'arêtes par requête (par exemple 5 000 arêtes) avec indicateur de troncature ; arrêt sur entités de service (exchanges, bridges, routeurs) ; agrégats d'arêtes précalculés par paire d'adresses et par token ; calculs lourds en tâche asynchrone avec résultat mis en cache. Aujourd'hui, chaque événement relit et recopie tout l'historique de l'adresse (`InMemoryFlowStore` recopie la liste à chaque lecture) : sur une adresse « hub » de 106 flux, l'ingestion temps réel devient impossible (HYPOTHÈSE fondée sur la complexité du code, non mesurée).

10. Audit du Programmable Alerting Engine

C'est le composant le plus abouti, mais sa chaîne de bout en bout ne fonctionne pas hors démo.

Capacité	Statut	Commentaire
DSL, YAML/JSON, validation	Implémenté+testé	Propriétés inconnues rejetées, chemins d'erreur JSON, tags et alias YAML refusés ; trois dialectes concurrents dans le dépôt.
Fenêtres temporelles	Implémenté+testé	Glissantes en temps d'événement, montants exacts ; pas de fenêtres fixes ; une seule agrégation par règle.
Corrélation	Partiel	Séquences ordonnées sur une seule clé ; impossible d'exprimer « A→B puis B→C » ou une absence d'événement.
Simulation	Implémenté+testé	Rejoue un tampon mémoire de 100 000 événements ; perdu au redémarrage.
Déduplication, cooldown	Implémenté+testé	Le cooldown regroupe au lieu de supprimer (correct) ; TTL de déduplication en temps réel et non en temps d'événement.
Retry, dead letter	Implémenté+testé	Aucun endpoint pour relancer les dead letters.
Escalade	Partiel	Hausse de priorité sans re-notification.
Versionnement	Partiel	Versions immuables en mémoire ; un `PUT` réinitialise l'état des fenêtres (détections perdues pendant l'édition).
Webhooks signés, secrets	Partiel	HMAC-SHA256 correct ; secret en clair en mémoire ; chiffrement annoncé (ADR-0009) absent.
Multi-tenancy	Absent	—

Les six règles cibles

Règle souhaitée	Exprimable	Comment ou écart
Plus de 100 000 USDT reçus en 10 minutes	**Oui**	Agrégation SUM groupée par destinataire, filtre sur le contrat (validé et testé par exécution).
Interaction avec une adresse à haut risque	**Partiel**	OR de groupes sur `counterparty.risk_score` et labels ; dépend de labels réels inexistants.
Franchissement d'un seuil de risque	**Non**	`RISK_SCORE_CHANGED` jamais émis ; la règle de niveau déclenche à chaque transfert.
Wallet dormant redevenu actif	**Partiel**	Via le signal `DORMANT_REACTIVATION`, seuil non paramétrable dans la règle.
Adresse surveillée vers un nouveau wallet	**Approximatif**	`counterparty.age_days == 0` signifie « vu pour la première fois par la plateforme » : avalanche de faux positifs après redémarrage.

Groupe de wallets coordonné	**Partiel**	DISTINCT_COUNT sur une liste ; pas de clustering ni de corrélation multi-entités.

Améliorations prioritaires (RECO). Identifiants de connecteurs compatibles avec le validateur et test bout en bout règle → webhook → livraison signée ; un seul dialecte (`openchain.io/rule/v1`) partout ; watchlists référencées par nom ; événement `RISK_SCORE_CHANGED` avec valeur précédente ; séquences multi-clés ; catalogue de champs restreint à ce qui est réellement alimenté ; persistance des règles, de l'état et des livraisons.

11. Audit de l'architecture

Sujet	Constat
Modularité	Bonne : `oci-core` sans dépendance, domaine sans Spring, seul `oci-app` assemble (ADR-0001 respecté).
Séparation des étapes	Ingestion, intelligence, scoring et alerting sont séparés en modules, mais enchaînés par appels directs synchrones dans un même verrou.
Concurrence	Sérialisation grossière (`synchronized` sur l'ingestion, le moteur de règles et les services d'alertes) ; un pool de 2 threads partagé entre minage, indexation, livraisons (jusqu'à 200 × 10 s de timeout) et escalade : un endpoint lent bloque l'ingestion.
Événementiel	Kafka/RabbitMQ et outbox : ADR seulement ; `OCI_ROLES=api,worker` jamais lu.
Stockage	Aucun. PostgreSQL inutilisé ; ClickHouse et Apache AGE : ADR seulement ; `installation.md` mentionne des migrations Flyway inexistantes.
Idempotence	Ingestion et livraisons : oui ; API (`Idempotency-Key`) : non.
Observabilité	Aucune métrique exportée ; Grafana interroge des métriques inexistantes ; collecteur OpenTelemetry sans émetteur.
Extensibilité multi-chaînes	SPI correct ; ajouter un EVM est peu coûteux ; Bitcoin et Solana demandent un modèle de flux propre.

Technologies de stockage

Technologie	Usage adapté	Avis
PostgreSQL	Règles, alertes, clés, audit, labels, checkpoints, outbox ; flux partitionnés par mois et réseau	**Socle unique recommandé, dès maintenant**
Apache AGE	Requêtes de graphe (openCypher) dans PostgreSQL	Plus tard, si les requêtes récursives SQL ne suffisent pas ; Apache-2.0
ClickHouse	Historique massif de transferts, agrégats analytiques	Quand le volume le justifie (plusieurs réseaux complets) ; Apache-2.0
Redis	Cache, rate limiting distribué	Non nécessaire tant qu'il n'y a qu'une instance
OpenSearch	Recherche plein texte	Non justifié aujourd'hui
Neo4j	Graphe natif	Déconseillé : Community en GPL-3.0, clustering réservé à l'Enterprise
MongoDB	Documents	Aucun besoin qui ne soit couvert par PostgreSQL (JSONB)
Kafka	Découplage ingestion / traitement à grande échelle	Après l'outbox PostgreSQL, quand plusieurs workers sont nécessaires

Passerelle WSO2 (demandée en parallèle). La vérification des faits sur WSO2 API Manager 4.7 montre que les images publiques ne reçoivent pas de correctifs de sécurité sans abonnement, et que l'intégration demande au moins 4 Go de RAM et un travail significatif (JWT backend, mTLS, API Products). RECO : la placer en phase 4 (API

B2B), après la persistance et la fiabilité des données, pour ne pas industrialiser l'exposition d'une API dont les données ne sont pas encore fiables.

12. Audit de sécurité

Vérifications exécutées (application démarrée, profil démo)

Test	Résultat observé
Sans clé / clé invalide	**401 problem+json**
Clé révoquée	**401 « invalid, expired or revoked API key »**
Permission insuffisante (`alerts:read` sur le risque ; clé UI sur webhooks et clés)	**403**
Clé passée en paramètre d'URL	**400**
Secret dans la liste des clés	**Absent**
Webhook vers 127.0.0.1	**Bloqué par la garde SSRF**
Route inconnue	**404 hors format problem+json (sans `type` ni `traceId`)**

Faiblesses

Gravité	Constat	RECO
Élevée	Notifications inopérantes hors démo : détection silencieuse, escalades sans destinataire	Corriger les identifiants, test bout en bout, alerte sur dead letters
Élevée	Aucune persistance : décisions des analystes, règles et clés perdues au redémarrage ; un OOM (`ExitOnOutOfMemoryError`) efface tout	PostgreSQL + migrations
Élevée	Score manipulable par dusting ou faux token	Filtre tokens connus, seuils en valeur, détecteur de dusting
Moyenne	Pas de rate limiting ; `/rules/validate` (lecture seule) exécute jusqu'à 50 tests × 10 000 événements ; les transactions d'un wallet sont relayées vers le RPC avec la clé de l'opérateur	Token bucket par clé, budgets dédiés
Moyenne	Journal d'audit limité à un logger, incomplet (création, modification et import de règles, transitions d'alertes non tracés)	Table append-only
Moyenne	Stockages mémoire non bornés : déni de service possible par un analyste	Bornes, rétention, quotas
Moyenne	Rôles utilisateurs sans effet sur l'autorisation (`UserDirectory`)	Brancher ou retirer l'écran
Moyenne	Jeton du frontend en `localStorage`	OIDC PKCE en mémoire ou cookie httpOnly
Moyenne	Compose : mode démo par défaut, port 8080 sur toutes les interfaces, clé admin imprimée dans les logs	Démo opt-in, liaison 127.0.0.1
Moyenne	URL complète des webhooks visible avec `webhooks:read` (rôle VIEWER) : fuite de jetons intégrés aux URL	Masquer chemin et paramètres
Faible	Swagger public, pepper éphémère si non configuré, `PUT` sans If-Match, URL RPC en http acceptée	Durcissement de la configuration de production
Info	Multi-tenancy décrit dans la doc mais absent	Retirer de la doc ou implémenter

Conformité. ChainSentinel n'est ni certifié ni « conforme » à un texte : aucune preuve ne permet de l'affirmer. Il doit se présenter comme un outil produisant des indicateurs d'analyse et des signaux de risque, jamais des accusations.

La console affiche déjà un avertissement (« Signals indicate risk indicators or anomalies, not wrongdoing ») ; il doit figurer aussi dans les exports et les webhooks.

13. Données et threat intelligence

Source	Licence / conditions	Coût	Fréquence	Fiabilité, couverture, risques
OFAC SDN (adresses)	Œuvre du gouvernement US, en principe hors droit d'auteur ; termes de redistribution à vérifier	0	À chaque désignation	Autorité primaire ; listes non exhaustives selon l'OFAC ; format XML susceptible de changer
Gels USDT / USDC on-chain	Données publiques	Coût RPC	Temps réel	Faits on-chain fiables ; nom exact de l'événement USDC à vérifier sur l'ABI
GraphSense TagPacks	MIT	0	Actif	Provenance obligatoire ; couverture modeste
UK OFSI, liste UE	À vérifier	0	Ad hoc	Très peu d'adresses
ScamSniffer	GPL-3.0	0 (API temps réel payante)	Quotidien, délai 7 jours	Domaines seulement ; fichier d'adresses non mis à jour depuis 02/2024 ; à télécharger à l'exécution, pas à redistribuer
MetaMask eth-phishing-detect	DBAD 1.2 (non OSI)	0	Très fréquent	Domaines ; licence atypique
Etherscan labels	Conditions interdisant la reproduction des labels et l'usage pour constituer des datasets	—	—	**Interdit**
Dune Spellbook	BSL 1.1, usage interdit pour une « Data or Analytics Platform » jusqu'au 3 mars 2027	—	—	**Interdit jusqu'à cette date**
OpenSanctions	CC BY-NC : licence commerciale requise, même en interne	Payant	Quotidien	Bonne normalisation ; coût
Chainabuse (TRM)	Clé standard : 10 appels par mois ; conditions de réutilisation à vérifier	0 limité	Continu	Signalements non vérifiés ; dépendance forte
APIs sanctions gratuites (TRM, Chainalysis, Scorechain)	Conditions propriétaires	0	Temps réel	Gratuité révocable ; TRM : 100 requêtes/jour sans clé

Modèle de label recommandé. Ajouter à `AddressLabel` : licence, version ou empreinte du fichier source, date d'observation obligatoire, date de vérification, date de validité, statut (actif, délisté, contesté), historique des changements ; permettre la suppression et le délistage (impossible aujourd'hui : `LabelRepository` n'a pas de suppression) ; valider le format des adresses au chargement (une adresse TRON en hexadécimal ne correspond aujourd'hui jamais à l'adresse Base58 indexée, sans erreur).

14. Analyse UX/UI

Parcours	État	Frictions et manques

Recherche d'un wallet	Implémenté	Les adresses `0x…` sont ambiguës entre Ethereum, BSC et Polygon.
Consultation d'une transaction	Implémenté	—
Exploration du graphe	Lecture seule	Ni sauvegarde, ni annotation, ni export ; graphe trop dense en démo.
Analyse d'une alerte	Partiel	Les preuves d'agrégation (valeur, seuil, fenêtre) ne sont pas affichées ; l'événement apparaît en JSON brut ; pas de traitement en lot.
Création d'une règle	**Cassé**	Le Rule Builder produit un format refusé (en cours de correction) ; seule l'édition YAML manuelle fonctionne.
Simulation	Partiel	Fonctionne pour les règles enregistrées seulement.
Investigation d'incident	Absent	Pas de dossier, pas de lien alerte → graphe préfiltré.
Export de rapport	Absent	Seul l'export YAML des règles existe.
Sources de renseignement	Absent	Labels chargés par fichier au démarrage, sans écran ni API.
API et intégrations	Partiel	Clés API fonctionnelles ; l'écran propose Slack, Discord, Telegram et email, refusés par le backend.

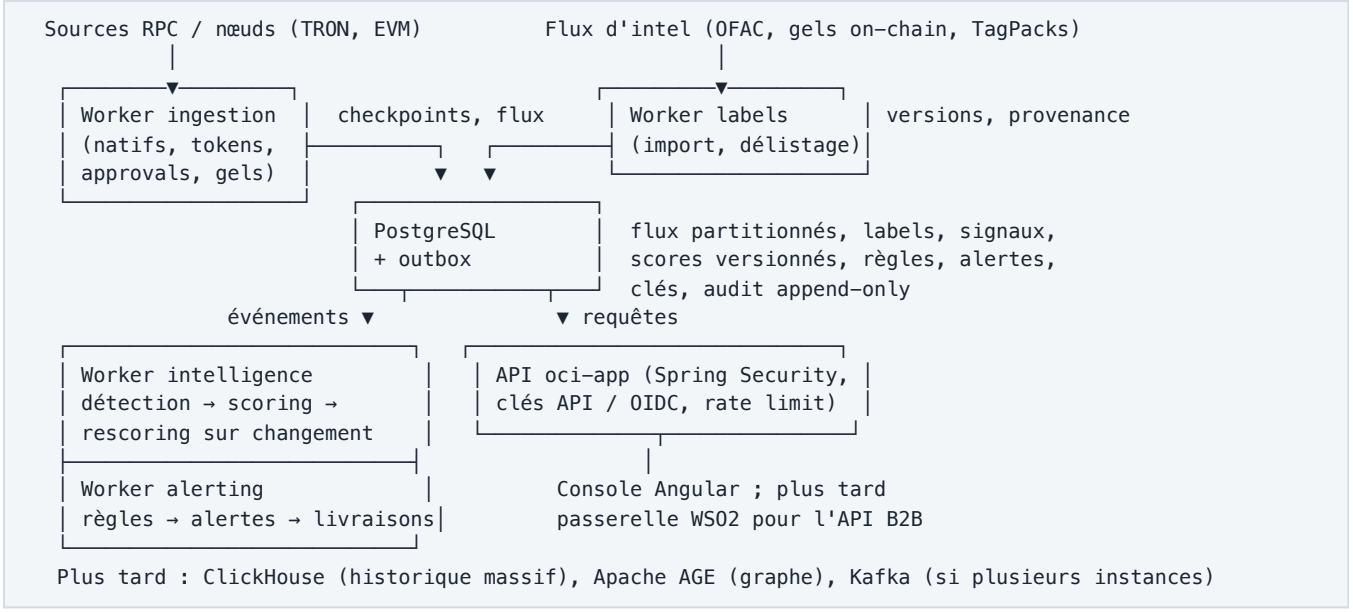
Indispensable (RECO). Bandeau permanent quand les données sont simulées, et mock exclu des builds de production ; vue d'alerte centrée sur l'explication (règle, valeur observée, seuil, contributions au score, chemin d'exposition) ; dossier d'investigation reliant alertes, adresses, graphe et notes, exportable en PDF ; désactivation explicite, avec explication, de toute action non supportée par le backend.

15. Open source et modèle économique

Option	Avantages	Inconvénients	Exemples
Apache-2.0 (actuel)	Adoption maximale, clause de brevets, compatible AGE, ClickHouse, Cryo, reth	Revente en SaaS possible sans contrepartie	GraphSense (MIT) + offre commerciale Iknaio
AGPL-3.0	Protège du SaaS parasite, reste OSI	Rebute certaines banques ; CLA nécessaire pour double licence	OpenZeppelin Monitor, Grafana (2021)
BSL / ELv2	Protection forte	Pas open source ; risque de fork et de perte de confiance	HashiCorp → fork OpenTofu
Open core	Cœur ouvert, modules payants	Frontière payant / gratuit à gouverner	Neo4j, Elastic

RECO. Garder Apache-2.0 pour le cœur : sans données propriétaires, la valeur du projet passe par l'adoption. Monétiser par : édition self-hosted entreprise (multi-tenancy, SSO, rapports réglementaires, haute disponibilité), packs de règles et de données maintenus (sanctions, gels, typologies calibrées), connecteurs premium (SIEM, case management, Travel Rule), hébergement géré, support. Ne pas changer de licence sans décision explicite : un passage à l'AGPL protégerait d'un hébergeur concurrent mais freinerait les intégrateurs bancaires. Le DCO est déjà en place (`CONTRIBUTING.md`) ; un CLA ne serait nécessaire qu'en cas de double licence future. Ne lier aucun code AGPL ou GPL (OZ Monitor, TrueBlocks, Neo4j Community) au binaire Apache.

16. Architecture cible recommandée



Principes : un seul binaire et plusieurs rôles (`api`, `ingest`, `intel`, `alerting`) activés par configuration, déployables séparément ; PostgreSQL unique au départ ; outbox transactionnelle avant Kafka ; agrégats incrémentaux au lieu de rebalayer l'historique ; observabilité Micrometer + Prometheus + OpenTelemetry dès la persistance.

17. Matrice des priorités P0 à P3

Fonctionnalité	Problème résolu	Valeur métier	Complexité	Dépendances	Priorité
Persistance PostgreSQL + migrations	Perte totale au redémarrage	Prérequis à tout usage réel	L	—	**P0**
Livraison réelle des notifications + test bout en bout	Alertes jamais reçues	Cœur de la proposition	S	—	**P0**
Durcissement de l'exposition (tokens connus, seuils, anti-dusting)	Score manipulable	Crédibilité	M	Registre de tokens	**P0**
Un seul dialecte de règles partout	Règles refusées	Adoption	S	—	**P0**
Ingestion des natifs, approvals et gels USDT/USDC	Données incomplètes	Signaux à forte valeur	M	Persistance	**P0**
Sources réelles : OFAC + gels + TagPacks avec provenance	Aucune intel réelle	Valeur immédiate	M	Modèle de label	**P0**
Documentation alignée sur le code	Promesses non tenues	Confiance	S	—	**P0**
Contrat ADR-0008 (plafond par preuve, direct > indirect, rescoring)	Scores trompeurs	Explicabilité	M	Persistance	**P1**

Rate limiting, bornes mémoire, audit append-only	DoS, traçabilité	Production	M	Persistance	**P1**
Observabilité réelle	Exploitation à l'aveugle	Production	S	—	**P1**
Rétractations de reorg dans l'intelligence	Signaux fantômes	Intégrité	M	Persistance	**P1**
Séparation des pools ingestion / livraisons	Blocage en tête de file	Fiabilité	S	—	**P1**
Fixtures enregistrées depuis de vrais nœuds, tests BSC/Polygon	Adaptateurs non prouvés	Qualité des données	M	—	**P1**
Calibration des détecteurs et exclusions d'entités de service	Faux positifs	Précision	L	Jeu de données réel	**P1**
Vue d'alerte explicable, bandeau données simulées	Triage aveugle	UX analyste	S	—	**P1**
Fan-in, dusting, address poisoning	Typologies absentes	Différenciation	M	Ingestion complète	P2
`RISK_SCORE_CHANGED`, watchlists, séquences multi-clés	Règles inexprimables	Différenciation	M	Persistance	P2
Exposure score quantitatif (taint pro-rata)	Exposition non proportionnelle	Différenciation	L	Graphe persistant	P2
Dossiers d'investigation et export PDF	Pas de livrable d'enquête	Analystes, LE	M	Persistance	P2
L2 EVM (Arbitrum, Base, Optimism) et Avalanche	Couverture	Marché	M	Couche EVM fiable	P2
Connecteurs Slack, email, SIEM	Intégration	B2B	S	Livraison réelle	P2
Clustering heuristique	Entités non reliées	Investigation	L	Graphe persistant	P3
Multi-tenancy, SSO, passerelle API (WSO2)	B2B multi-clients	Édition entreprise	L	Persistance, rate limiting	P3
Bitcoin, puis Solana	Couverture	Marché	XL	Nouveau modèle de flux	P3
ClickHouse, Kafka	Volume	Échelle	L	Mesures de charge	P3

18. Feuille de route

Effort relatif : S (jours), M (1 à 2 semaines), L (3 à 6 semaines), XL (plus de 6 semaines), pour une petite équipe.

Phase	Objectifs et fonctionnalités	Modules et architecture	Tests et critères d'acceptation	Risques / effort

1. MVP crédible	Persistance ; livraison réelle des notifications ; un seul DSL ; exposition durcie ; natifs, approvals, gels ; OFAC + gels + TagPacks ; doc alignée	Tous ; PostgreSQL + migrations ; rôles worker	Redémarrage sans perte ; test bout en bout règle → webhook signé ; dusting sans effet sur le score ; fixtures réelles TRON/ETH	Glissement de périmètre ; **L à XL**
2. Plateforme d'investigation	Contrat ADR-0008, rescoring, exposure score ; dossiers, export PDF ; vue d'alerte explicable ; graphe persistant	intelligence, app, frontend ; agrégats d'arêtes	Chaque score reproductible et versionné ; graphe 2 sauts < 2 s sur hub (à mesurer)	Performance des graphes ; **L**
3. Détection avancée	Fan-in, dusting, address poisoning, layering ; calibration ; exclusions d'entités de service ; séquences multi-clés	intelligence, alerting	Précision et rappel publiés par détecteur sur jeu étiqueté	Accès à des données étiquetées ; **L**
4. API et intégrations B2B	Rate limiting, idempotence, ETag ; OIDC complet ; connecteurs Slack, email, SIEM ; passerelle WSO2 ; L2 EVM	app, déploiement	Tests de contrat OpenAPI ; tests de charge ; sécurité de la passerelle (mTLS, JWT)	Images WSO2 non corrigées sans abonnement ; **L**
5. Industrialisation et écosystème	Multi-tenancy, HA, SBOM et scan d'images, packs de règles communautaires, programme de contributeurs, Bitcoin	Tous ; ClickHouse/Kafka si mesuré	Installation depuis zéro documentée et testée en CI ; montée de version sans perte	Charge de maintenance ; **XL**

19. Risques majeurs

Risque	Probabilité	Impact	Mitigation
Résultats faux présentés comme fiables (faux positifs, dusting)	Élevée	Élevé (réputation, juridique)	P0 sur l'exposition, plafonds par niveau de preuve, avertissements systématiques
Écart documentation / code perçu comme trompeur	Élevée	Élevé (confiance OSS)	Réécrire le README sur l'existant, statut par fonctionnalité
Dispersion : trop de chantiers en parallèle (WSO2, nouvelles chaînes, UI)	Élevée	Élevé	Geler le périmètre sur la phase 1
Dépendance à des RPC tiers (quotas, coûts)	Moyenne	Moyen	Failover branché, budgets par réseau, nœuds self-hosted
Données d'intel sous licence restrictive intégrées par erreur	Moyenne	Élevé (juridique)	Champ licence obligatoire, liste de sources autorisées, revue
Concurrence OSS (OpenZeppelin Monitor)	Moyenne	Moyen	Différenciation : scoring explicable, TRON/USDT, stablecoins
Scalabilité sur adresses « hub »	Élevée en réel	Élevé	Agrégats incrémentaux, tests de charge

20. Recommandation finale et dix décisions

> **Verdict : continuer, en repositionnant et en simplifiant.** Pas de réécriture : l'architecture modulaire et le moteur de règles sont de bonne qualité et supportent une amélioration progressive. Il faut repositionner (surveillance programmable et scoring explicable self-hosted, pas « AML complet ») et simplifier (geler les nouvelles chaînes, la passerelle et les écrans tant que les données et la persistance ne sont pas fiables).

Si je devais investir les six prochains mois, les dix décisions les plus importantes seraient :

1. **Positionner le produit** comme « surveillance on-chain programmable et scoring explicable, self-hosted », centré sur TRON, Ethereum et les stablecoins, et le dire clairement dans le README.
2. **Rendre PostgreSQL obligatoire** dès maintenant, avec migrations versionnées et outbox ; abandonner tout fonctionnement « en mémoire » hors démo.
3. **Garantir la chaîne de bout en bout** règle → alerte → notification signée par un test d'intégration bloquant en CI.
4. **Compléter l'ingestion** de TRON et Ethereum (natifs, transactions internes, approvals, gels USDT/USDC) avant d'ajouter le moindre réseau.
5. **Rendre le score inattaquable** : filtre des tokens inconnus, seuils en valeur, anti-dusting, plafond par niveau de preuve, quatre scores distincts, rescoring sur changement de label.
6. **Publier un pack de données ouvert et sourcé** (OFAC, gels on-chain, TagPacks) avec licence, provenance et historique par label, et une liste blanche de sources juridiquement utilisables.
7. **Imposer un seul DSL** (`openchain.io/rule/v1`) et un schéma partagé par le backend, le frontend et la documentation, vérifié par un test de contrat.
8. **Mesurer la qualité** : jeu de cas réels étiquetés, précision et rappel publiés par détecteur, fixtures enregistrées depuis de vrais nœuds.
9. **Construire l'expérience analyste** autour de l'explication : vue d'alerte détaillée, dossier d'investigation, export PDF, bandeau permanent quand les données sont simulées.
10. **Garder Apache-2.0 et préparer le modèle open core** (édition entreprise, packs de règles et de données, support), en reportant la passerelle WSO2 et la multi-tenancy à la phase 4.

Limites de cet audit : les travaux de la phase 2 non committés n'ont pas été évalués ; les adaptateurs n'ont pas été testés contre de vrais nœuds ; OIDC n'a pas été testé avec un fournisseur d'identité ; aucun test de charge n'a été exécuté ; Docker Compose n'a pas été démarré ; les informations de marché proviennent de pages publiques et peuvent relever du marketing (signalé comme tel). Rapports de travail détaillés : `01-chain-detection-scoring.md`, `02-alerting-api-security-ux.md`, `03-market-competition-data.md`.